SOLAR PV FAULT DETECTION AND CLASSIFICATION USING AN ARTIFICIAL NEURAL NETWORK

A PROJECT REPORT

Submitted by

Navya Khandelwal

Reg No: RA2411026030021

Under the guidance of

Dr. Chiranjit Dutta

(Department of Computer Science & Engineering)



SRM INSTITUTE OF SCIENCE & TECHNOLOGY DELHI-NCR CAMPUS,
GHAZIABAD (U.P.)

ACADEMIC YEAR 2026–2027

SRM INSTITUTE OF SCIENCE & TECHNOLOGY

(Under Section 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified to be the bonafide record of work done by ***Navya Khandelwal (RA2411026030021)*** *of* ***B.TECH*** *degree course in* ***SRM INSTITUTE OF SCIENCE AND TECHNOLOGY, DELHI-NCR CAMPUS*** *of Department of Computer Science & Engineering, for the project titled "SOLAR PV FAULT DETECTION AND CLASSIFICATION USING AN ARTIFICIAL NEURAL NETWORK", under the guidance of* ***Dr. Chiranjit Dutta****, during the academic year 2026-2027.*

SIGNATURE

Dr. Chiranjit Dutta
Faculty Guide
Computer Science & Engineering

SIGNATURE

Dr. Neelam Singh
Head of the Department
Computer Science & Engineering

Table of Contents

S.N.	Title	Page No.
1	ABSTRACT	4
2	INTRODUCTION	5
3	EXISTING PROBLEMS	7
4	PROBLEMS SOLUTIONS	9
5	TECHNOLOGIES USED	11
6	ARCHITECTURAL DIAGRAM	13
7	DATASET DESCRIPTION	16
8	METHODOLOGY	19
9	RESULTS	22
10	CONCLUSIONS	25
11	REFERENCES	27

ABSTRACT

This project develops a supervised artificial neural network (ANN) to classify photovoltaic observations into Normal, Short Circuit, Degradation, Open Circuit, and Shadowing classes. Six measurements are used: irradiance, PV temperature, two DC current channels, and two DC voltage channels.

The supplied script loads electrical and ambient variables from MATLAB files, combines them into a table, samples 5,000 observations from each class, and forms 25,000 balanced rows. It uses a stratified 80/20 train-test split, fits StandardScaler on training data, and trains a feedforward network with 32- and 16-unit ReLU layers and a five-unit softmax output. Training uses 20 epochs, batch size 64, and a 20% validation split from the training partition.

The project presentation reports 98.54% accuracy on a balanced 5,000-row test set (4,927 correct) and rounded class metrics. These are presentation-reported values; exact confusion-matrix cells and saved evaluation logs were not supplied. Resampling before splitting can place repeated observations in both partitions, so an evaluation with splitting before balancing is recommended.

Keywords: photovoltaic systems, fault detection, multiclass classification, artificial neural network, class imbalance, StandardScaler.

INTRODUCTION

1. Background and Motivation

Photovoltaic systems convert solar radiation into electrical power. Their output changes with environmental conditions and operating state, so a single voltage or current threshold may not describe system health. Multiple measurements offer a richer signature, but relationships among readings and operating conditions can be nonlinear.

This project treats fault identification as supervised classification. A model receives six measurements and returns one of five labels. It is a decision-support function that maps patterns represented in training data to a predicted class. It does not isolate equipment, determine root cause, or replace engineering inspection.

2. Problem Statement

Given labelled PV observations containing irradiance, temperature, two DC current channels and two DC voltage channels, train and save a model that predicts one of five operating classes. Training and inference must use the same feature ordering and scaling.

3. Objectives

The objectives of the project and the implementation evidence that supports each of them are summarised below.

Objective	Evidence
Prepare model-ready data	MATLAB variables become a seven-column DataFrame
Address label imbalance	5,000 samples drawn per class
Train a multiclass ANN	Keras Sequential model with softmax output
Evaluate classification	Presentation reports test metrics
Support single-record prediction	predict.py loads model and scaler

Table 1: Project objectives and evidence

4. Scope and Limitations

This report covers data preparation, model definition, training configuration, reported evaluation, and example inference. It concerns a five-class fixed-width input problem. Sensor

acquisition, online streaming, fault localization, power-loss estimation, plant controls, and a graphical interface are outside scope.

The code assumes variables irr, pvt, idc1, idc2, vdc1, vdc2 and f_nv. It assumes labels 0-4 correspond in order to Normal, Short Circuit, Degradation, Open Circuit and Shadowing. Confirm this mapping against dataset documentation before operational use.

Evaluation uses a balanced test partition, not one preserving original class proportions. Thus accuracy is not expected field accuracy. The scripts do not implement input validation, calibrated probabilities, alert thresholds or model versioning.

5. Report Organization

The Abstract summarises the work. This Introduction states the problem, objectives and scope. Existing Problems and Problems Solutions describe the weaknesses of simple monitoring and of the data pipeline, and how the project addresses them. Technologies Used lists the software stack and source files. The Architectural Diagram section presents the workflow and the network design. Dataset Description and Methodology explain the data, preprocessing, training and evaluation procedure. Results summarises the reported performance, Conclusions closes the report with limitations and future work, and References lists the sources.

EXISTING PROBLEMS

1. PV Monitoring and Fault Classification

PV monitoring combines environmental and electrical observations to understand system performance. Irradiance and temperature describe operating context; current and voltage channels describe electrical response. Classification asks which predefined operating label best matches a measured feature vector.

Data-driven models can learn interactions that are difficult to encode with independent thresholds. Usefulness depends on labelled-data quality and coverage. If a class is missing, mislabeled, or represented under narrow conditions, generalization to those cases is uncertain.

2. Problems in the Data and Evaluation Pipeline

a) Severe Class Imbalance

The project presentation reports 84.65% Normal, 0.44% Short Circuit, 0.75% Degradation, 0.44% Open Circuit and 13.72% Shadowing. Normal operation dominates. Direct training on this distribution could favour the majority class while missing rare faults.

b) Changed Class Prevalence After Balancing

Equal counts simplify class comparisons and reduce majority dominance, but change class prevalence. Accuracy on this sample does not represent performance at natural fault frequencies. Probabilities trained on balanced data may need calibration or adjustment for deployment prevalence.

c) Duplicate Rows and Possible Test Leakage

The resampling function uses replacement by default. If a class has fewer unique rows than requested, duplicates are generated. Since sampling precedes train_test_split, copies can appear in both partitions. This may make test performance optimistic; the source material does not quantify the overlap.

d) Correlated Observations

The supplied files show no time or installation identifier, so group separation cannot be verified. If observations are correlated, random row splitting may put similar operating points in both partitions. A temporal or site-based holdout is recommended.

3. Limitations of the Current Implementation

- The scripts do not implement input validation, calibrated probabilities, alert thresholds or model versioning.
- Units, sensor calibration, sample interval, and channel placement are not stated in code and should be confirmed from dataset documentation.
- The code relies on relative paths and unpinned dependency versions, and the class IDs are assumed to match the class names.
- The script combines exploration, training, evaluation and plotting, and it displays plots interactively without writing metric tables or plots to files.
- There is no sensor integration and no explanation of individual predictions.

PROBLEMS SOLUTIONS

1. Supervised Multiclass Classification

Fault identification is treated as supervised classification. Let x be a vector of six measurements and y a label in {0,1,2,3,4}. Training estimates a mapping from x to class scores. The predicted class is the index with the largest output. This implementation uses integer labels with sparse categorical cross-entropy for mutually exclusive classes.

2. Artificial Neural Network Classifier

A feedforward ANN composes affine transformations and nonlinear activations. Hidden layers can represent interactions among inputs; the final softmax layer converts class scores into normalized outputs. Flexibility can model nonlinear boundaries, while validation helps detect patterns that do not persist beyond training data (Bishop, 2006; Goodfellow et al., 2016). The compact network used here represents nonlinear interactions with few parameters.

3. Class Balancing

main.py creates one subset per label and calls sklearn.utils.resample for 5,000 samples with random_state=42. Subsets are concatenated and shuffled with the same seed, producing 25,000 rows with 5,000 per class. This prevents the majority class from dominating training.

4. Stratified Split and Feature Scaling

The code selects the six sensor variables as X and f_nv as y. train_test_split reserves 20% for testing, sets random_state=42, and stratifies on y. StandardScaler estimates means and standard deviations from X_train only, transforms train and test data, and is saved as solar_pv_scaler.pkl.

5. Single-Record Prediction

predict.py loads the model and scaler, constructs a one-row NumPy array, transforms it, obtains class scores and returns the argmax label and maximum score as a percentage. The script saves solar_pv_ann.keras and solar_pv_scaler.pkl in the working directory. Both must be retained together because prediction must use the training scaler.

6. Limitations and Recommended Controls

The project demonstrates a prototype path from labelled sensor values to a saved multiclass model and prediction script. Its reported balanced-test performance is promising as a prototype, but its practical meaning depends on data provenance, sampling design, label definitions and independent evaluation. The main risks and the controls recommended for each are listed below.

Limitation or risk	Recommended control
Resampling before split; duplicates possible	Split first; balance training only
Balanced test differs from field prevalence	Natural-prevalence site holdout
No exact confusion matrix	Save full matrix and report
Class IDs assumed to match names	Confirm dataset label map
No sensor integration	Add monitored acquisition and validation
Softmax max called confidence	Calibrate and communicate uncertainty
Relative paths / unpinned versions	Package versions and explicit paths
No prediction explanation	Evaluate attribution with domain review

Table 2: Limitations and recommended controls

7. Development Priorities

First revise partitioning to prevent duplicate overlap and preserve realistic prevalence. Confirm units and definitions for every input and label. Persist a reproducible evaluation bundle with matrix, metrics, training history, versions and split identifiers. Only then connect to live sensors or maintenance decisions.

TECHNOLOGIES USED

1. Technology Stack

The project is implemented in Python using the following libraries, as used by the supplied code.

Component	Technology	Purpose
Data loading	SciPy (scipy.io.loadmat)	Reads the MATLAB dataset files
Data handling	pandas, NumPy	Builds the DataFrame and the one-row prediction array
Preprocessing and metrics	scikit-learn	Resampling, stratified split, StandardScaler, classification report and matrix
Model	TensorFlow / Keras	Sequential dense network, training and saving
Visualisation	Matplotlib, Seaborn	Training curves and confusion matrix plots
Persistence	joblib	Saves and loads the fitted scaler

Table 3: Technologies used in the project

2. Source Files and Implementation Map

The table maps each processing block to the file that implements it.

Block	File	Role
MAT loading and flattening	main.py	Loads ambient/electrical arrays
DataFrame construction	main.py	Combines six predictors and target
Class resampling	main.py	Creates 5,000 rows per label
train_test_split	main.py	Stratified 80/20 split
StandardScaler	main.py	Scales and saves transformer
Sequential model	main.py	Builds 6-32-16-5 ANN
fit/evaluate	main.py	Trains and evaluates
Classification report/matrix	main.py	Computes test diagnostics
model.save	main.py	Persists Keras model
load/transform/argmax	predict.py	Returns class and maximum score

Table 4: Source-file responsibilities

Sampling requests 5,000 per label with random_state=42. Replacement is not disabled. The scaler is fitted on X_train. Inference feature order must remain [irr, pvt, idc1, idc2, vdc1, vdc2].

3. Required Files and Environment

Training requires main.py and dataset_elec.mat and dataset_amb.mat. Prediction requires predict.py, solar_pv_ann.keras and solar_pv_scaler.pkl. The scripts expect these paths relative

to the working directory, and MAT files, model and scaler must be in the process working directory. SciPy, pandas, Matplotlib, scikit-learn, joblib, TensorFlow and Seaborn must be installed.

4. Training Steps

1. Confirm expected MAT variables and equal array lengths.
2. Run main.py with SciPy, pandas, Matplotlib, scikit-learn, joblib, TensorFlow and Seaborn installed.
3. Review dataset shape, class counts, model summary, metrics and plots.
4. Keep model and scaler together; record package and data versions.

5. Implementation Review

A robust release should specify paths, pin dependency versions, validate array shapes and finite values, verify label mapping, and handle missing or out-of-range inputs.

Component	Implementation	Review
Data loading	scipy.io.loadmat	Variable names assumed
Balancing	resample per class	Replacement can duplicate rows
Split	stratified 80/20	After balancing
Scaling	StandardScaler	Fit on train only
Classifier	Keras dense network	837 parameters
Evaluation	report and matrix	Printed, not persisted
Inference	saved model + scaler	No validation/calibration

Table 5: Implementation review summary

The source has a fixed feature list, stratified split, training-only scaler fit, and persists model artifacts. Its principal evaluation caveat is resampling before splitting. The script combines exploration, training, evaluation and plotting; separating these concerns would improve reproducibility.

ARCHITECTURAL DIAGRAM

1. End-to-End Workflow

The pipeline runs from the MATLAB data files to a saved model and scaler. Each stage and its output are listed below and illustrated in Figure 1.

Stage	Operation	Output
Load	Read ambient/electrical MAT variables	Flattened arrays
Assemble	Build table with predictors and target	Seven columns
Balance	Sample 5,000 per class	25,000 rows
Split	Stratified 80/20	20,000 / 5,000
Scale	Fit scaler on training data	Scaled inputs + scaler
Train	Keras ANN, 20 epochs	Trained model
Evaluate	Metrics and plots	Diagnostics
Save	Persist model	solar_pv_ann.keras

Table 6: End-to-end workflow stages

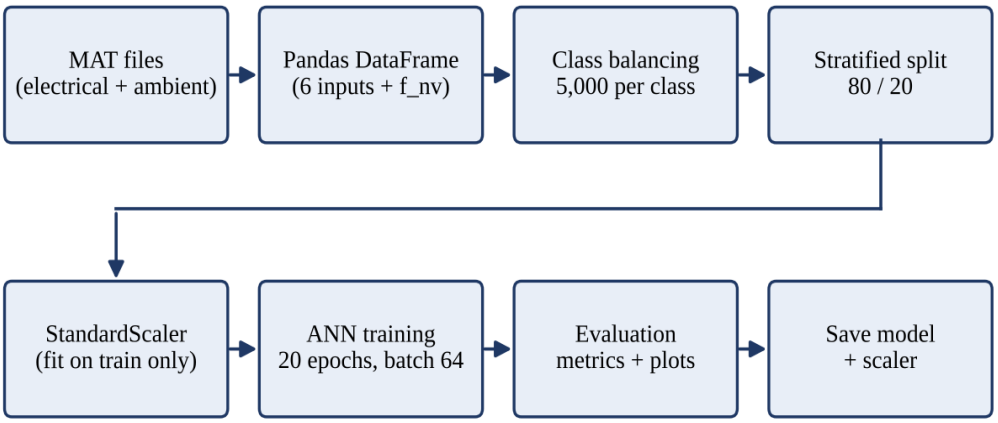


Figure 1: Data preparation and modelling workflow

The script displays plots interactively and saves model and scaler, but does not write metric tables or plots to files.

2. Neural Network Architecture

The Keras Sequential network expects six scaled numeric inputs. Dense hidden layers contain 32 and 16 ReLU units. The output layer has five softmax units, one per class. The layer structure is shown in Figure 2.

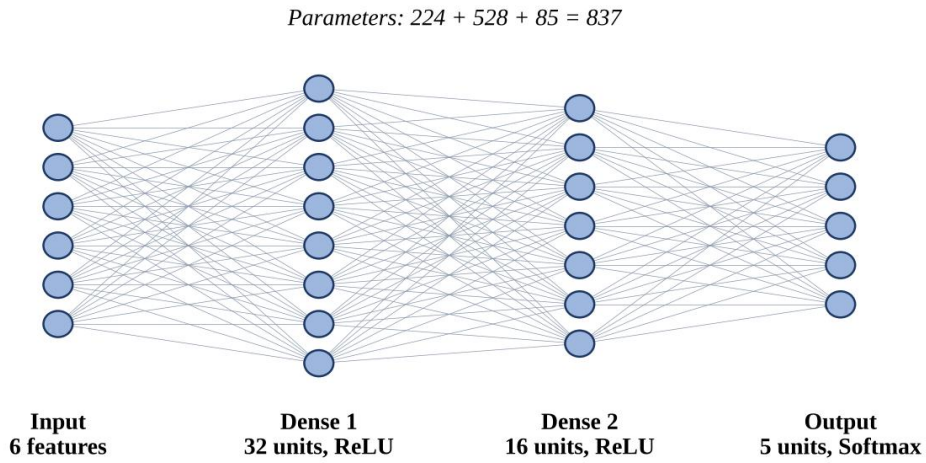


Figure 2: ANN layer structure: 6 inputs → Dense(32, ReLU) → Dense(16, ReLU) → Dense(5, Softmax) (hidden layers are drawn with fewer nodes than actual units)

Layer	Width	Parameters
Input	6	0
Dense 1, ReLU	32	224
Dense 2, ReLU	16	528
Dense output, softmax	5	85
Total	5	837

Table 7: Network dimensions and parameters

The parameter total includes one bias per dense unit: (6×32+32)+(32×16+16)+(16×5+5)=837, matching the project slide. Softmax outputs sum to one. predict.py calls the maximum output "Confidence"; it is a score and is not necessarily calibrated.

3. Inference Sequence

At prediction time a single measurement vector passes through the saved scaler and the saved network, as shown in Figure 3.

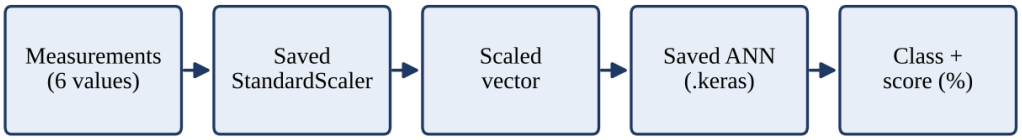


Figure 3: Inference sequence

4. Saved Artifacts

The script saves solar_pv_ann.keras and solar_pv_scaler.pkl in the working directory. Both must be retained together because prediction must use the training scaler.

DATASET DESCRIPTION

1. Data Sources and Schema

main.py loads dataset_elec.mat and dataset_amb.mat using scipy.io.loadmat, flattens arrays and combines them in a pandas DataFrame. The project presentation describes 1,373,798 original observations and seven fields: six inputs and one target.

Field	Meaning	Role
irr	Irradiance	Input
pvt	PV temperature	Input
idc1	DC current channel 1	Input
idc2	DC current channel 2	Input
vdc1	DC voltage channel 1	Input
vdc2	DC voltage channel 2	Input
f_nv	Fault label	Target

Table 8: Input feature definitions

predict.py supplies one observation in the same feature order. Units, sensor calibration, sample interval, and channel placement are not stated in code and should be confirmed from dataset documentation.

2. Target Classes

Index	Class	Description
0	Normal	Normal operation
1	Short Circuit	Short-circuit label
2	Degradation	Degradation label
3	Open Circuit	Open-circuit label
4	Shadowing	Shadowing label

Table 9: Target class definitions

3. Original Class Distribution

The project presentation reports the following class proportions in the original data. Normal operation dominates, so direct training on this distribution could favour the majority class while missing rare faults.

Class	Reported share of original data
Normal	84.65%
Short Circuit	0.44%
Degradation	0.75%
Open Circuit	0.44%
Shadowing	13.72%

Table 10: Reported original class distribution

4. Balanced Sample

main.py creates one subset per label and calls sklearn.utils.resample for 5,000 samples with random_state=42. Subsets are concatenated and shuffled with the same seed, producing 25,000 rows with 5,000 per class.

5. Balancing Implications

Equal counts simplify class comparisons and reduce majority dominance, but change class prevalence. Accuracy on this sample does not represent performance at natural fault frequencies. Probabilities trained on balanced data may need calibration or adjustment for deployment prevalence.

resample uses replacement by default. If a class has fewer unique rows than requested, duplicates are generated. Since sampling precedes train_test_split, copies can appear in both partitions. This may make test performance optimistic; the source material does not quantify overlap.

6. Training, Validation and Test Partitions

Training data updates weights; validation data monitors learning; test data is held out for final evaluation. The code reserves 20% for testing, then uses 20% of the remaining training partition for validation.

Partition	Rows	Note
Balanced dataset	25,000	5,000 per class
Training partition	20,000	80% of balanced data
Test partition	5,000	20% of balanced data
Validation (from training)	4,000	20% of training partition
Fitting rows	16,000	Training minus validation

Table 11: Intended row partitions

These counts describe row partitions. They do not ensure distinct source observations across partitions because resampling happens before splitting and uses replacement.

METHODOLOGY

1. Problem Formulation

Let x be a vector of six measurements and y a label in {0,1,2,3,4}. Training estimates a mapping from x to class scores, and the predicted class is the index with the largest output. Integer labels are used with sparse categorical cross-entropy for mutually exclusive classes.

2. Data Preparation

The data are loaded from the MATLAB files, assembled into a seven-column DataFrame, balanced to 5,000 rows per class (25,000 rows in total), and shuffled with a fixed seed of 42. The six sensor variables form X and f_nv forms y.

3. Split and Scaling

train_test_split reserves 20% of the rows for testing, sets random_state=42, and stratifies on y. StandardScaler estimates means and standard deviations from X_train only, transforms train and test data, and is saved as solar_pv_scaler.pkl so that inference uses the same scaling.

4. Training Configuration

Setting	Value	Purpose
Optimizer	Adam	Adaptive updates
Loss	Sparse categorical cross-entropy	Integer-label multiclass loss
Metric	Accuracy	Correct fraction
Epochs	20	Training passes
Batch size	64	Rows per update
Validation split	0.20	Subset of training partition
Seed	42	Sampling and split repeatability

Table 12: Training configuration

Keras takes validation data from arrays passed to model.fit. Since those arrays represent the 20,000-row training partition, validation uses about 4,000 and fitting about 16,000 rows. Test has 5,000 rows.

main.py plots training and validation accuracy and loss. The presentation describes accuracy around or above 98% and decreasing losses, but the numeric epoch series were not supplied.

5. Evaluation Metrics

A confusion matrix counts true and predicted classes. For class k, precision is TP/(TP+FP), recall is TP/(TP+FN), and F1 is their harmonic mean. Macro averages weight classes equally; weighted averages account for support. The project slide reports per-class metrics and support.

Metric	Meaning	Use
Accuracy	Correct / all predictions	Overall correctness
Precision	TP / predicted positives	Correctness of class assignments
Recall	TP / actual positives	Recovery of examples in class
F1-score	Harmonic mean of precision and recall	Balances precision and recall
Support	True examples per class	Context for metrics

Table 13: Evaluation metrics

6. Prediction Procedure

predict.py loads the model and scaler, constructs a one-row NumPy array in the fixed feature order, transforms it, obtains class scores and returns the argmax label and maximum score as a percentage. The example values below are hard-coded in the script and demonstrate the input format.

Position	Feature	Example
1	Irradiance	955.724
2	PV temperature	46.0895
3	DC current 1	9.1298
4	DC current 2	8.7981
5	DC voltage 1	259.089
6	DC voltage 2	201.761

Table 14: Prediction input order

A predicted class is not stated because the supplied files do not include the live output for this sample. To use the script, replace all six values while preserving order and units. Predictions require definitions and ranges compatible with training.

7. Recommended Evaluation Protocol

For stronger evaluation, split original records before oversampling. Use group-based partitions by time, array, site or acquisition session where identifiers exist. Balance only training data. Keep validation and test sets at natural prevalence and report per-class metrics. A stronger

evaluation should also preserve a natural-prevalence holdout, repeat analysis across seeds, and report a complete confusion matrix, uncertainty, dataset version and software versions.

8. Glossary and Abbreviations

Term	Definition
Balanced sample	Equal number of records per class
Batch size	Rows per gradient update
Feature scaling	Centering and scaling numeric inputs
Holdout	Data excluded from fitting for evaluation
Oversampling	Increasing class representation by resampling
Softmax	Output transformation whose values sum to one
Stratification	Partitioning that preserves class proportions
Test leakage	Overlap that weakens evaluation independence
Validation set	Data used to monitor training

Table 15: Glossary of key terms

Abbreviations: ANN: artificial neural network; DC: direct current; F1: harmonic mean of precision and recall; IoT: Internet of Things; PV: photovoltaic; ReLU: rectified linear unit; SHAP: SHapley Additive exPlanations.

RESULTS

1. Reported Test Performance

The presentation reports 4,927 correct classifications among 5,000 balanced test observations. This yields 98.54% accuracy.

Quantity	Value	Evidence
Balanced test set	5,000	Presentation
Correct	4,927	Presentation
Accuracy	98.54%	4,927 / 5,000
Incorrect	73	Derived by subtraction

Table 16: Reported test performance

The 73 errors are derived from the total. Exact confusion-matrix cells are not given in the extracted presentation text; the confusion matrix produced by the evaluation script is shown in Figure 4.

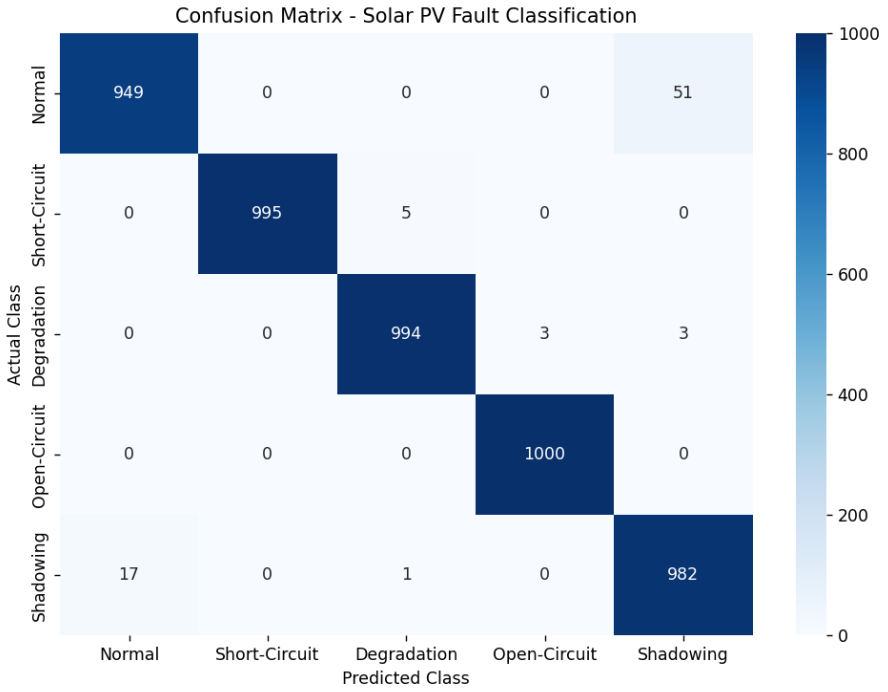


Figure 4: Confusion matrix – Solar PV fault classification

2. Training Curves

Slides describe training and validation accuracy rising to around 98% or higher and remaining close near training end. They also describe both losses decreasing. These are qualitative summaries, not epoch-by-epoch values. Validation accuracy is distinct from test accuracy.

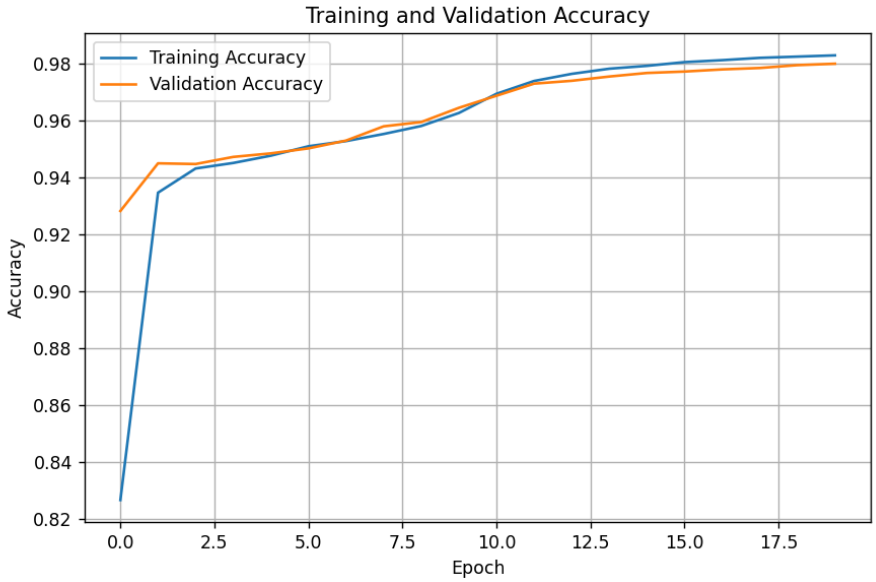


Figure 5: Training and validation accuracy

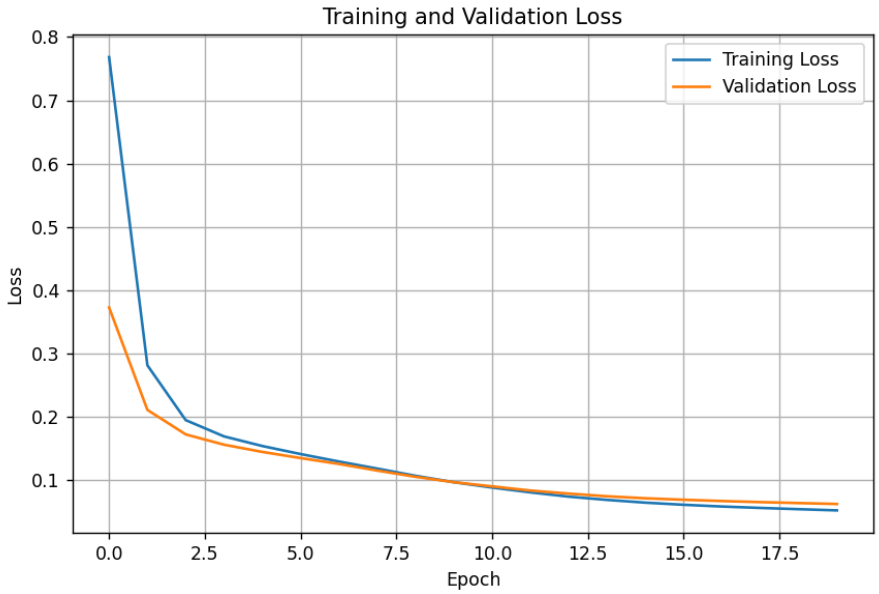


Figure 6: Training and validation loss

3. Class-wise Evaluation

The presentation gives per-class metrics rounded to two decimals and support of 1,000 for each class in the balanced test set.

Class	Precision	Recall	F1	Support
Normal	0.98	0.96	0.97	1,000
Short Circuit	1.00	1.00	1.00	1,000
Degradation	0.99	0.99	0.99	1,000
Open Circuit	1.00	1.00	1.00	1,000
Shadowing	0.95	0.98	0.97	1,000

Table 17: Reported class-wise results

Reported recall is lowest for Normal (0.96), and precision lowest for Shadowing (0.95). The direction and counts of specific confusions should be read from the confusion matrix rather than inferred from the rounded metrics.

4. Interpreting Metrics

Accuracy summarizes overall correctness. Per-class precision reflects false assignments to a class, while recall reflects missed examples. In PV maintenance, missed faults and unnecessary inspection may have different costs. A deployment evaluation should define those costs before setting thresholds.

5. Evaluation Conditions and Validity

The test set is balanced, so each class contributes one fifth of observations. The reported 98.54% summarizes this prepared sample; it is not expected accuracy at natural prevalence or a new PV site.

Because oversampling with replacement occurs before splitting, repeated observations may occur in both train and test partitions. Treat the slide metrics as reported outcomes, not independently reproduced or leakage-free field performance.

Evidence	Availability
Test accuracy	98.54% reported
Correct / total	4,927 / 5,000
Class metrics	Rounded to two decimals
Training curves	Plots and qualitative slide description
Confusion matrix	Plot from evaluation script; cell values not in slide text
Independent reproduction	Not performed

Table 18: Evidence available for evaluation

CONCLUSIONS

This project implements a supervised ANN for five PV operating classes using irradiance, temperature, two current readings and two voltage readings. The script balances data to 25,000 rows, makes a stratified 80/20 split, standardizes inputs, trains a 6-32-16-5 network, evaluates it and saves model and scaler. A second script demonstrates single-record inference.

The presentation reports 98.54% accuracy on a balanced test set of 5,000 observations with rounded class metrics. These results describe the prototype's prepared evaluation. Since balancing precedes splitting and the test set is balanced, further validation is needed before treating the score as field performance.

1. Current Limitations

- Resampling with replacement occurs before the train-test split, so duplicate rows may appear in both partitions.
- The test set is balanced and does not preserve natural class prevalence, so accuracy is not expected field accuracy.
- Class IDs are assumed to match the class names; units and sensor definitions are not stated in the code.
- No input validation, calibrated probabilities, alert thresholds, model versioning or prediction explanation is implemented.
- Evaluation results are printed and plotted but not persisted, and no independent reproduction has been performed.

2. Future Work

Future work should split before oversampling, test independent data from different periods or installations, assess natural prevalence, calibrate output scores, and evaluate false-negative costs. Later work can examine IoT monitoring, drift detection, interpretable diagnostics and early warning with domain validation.

3. Project Record

Record	Detail
Project	Solar PV fault classification using ANN
Student in supplied project materials	Navya Khandelwal, RA2411026030021
Inputs	irr, pvt, idc1, idc2, vdc1, vdc2
Target	f_nv; five classes
Balanced data	25,000 rows; 5,000 per class
Reported accuracy	98.54% (4,927 / 5,000)
Source files	main.py and predict.py

Table 19: Project record

The project presentation identifies Navya Khandelwal (RA2411026030021) as presenter. Implementation descriptions are attributed to the supplied code and performance values to the supplied project presentation.

REFERENCES

1. Bishop, C. M. (2006). Pattern recognition and machine learning. Springer.
2. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press.
3. Hastie, T., Tibshirani, R., & Friedman, J. (2009). The elements of statistical learning (2nd ed.). Springer.
4. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
5. Abadi, M., et al. (2016). TensorFlow: A system for large-scale machine learning. Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation, 265-283.